Smart Solve Calculations

Arithmetic Parser for Larger Compiler
Software Architecture Document

Version 1.0

Jaiden Green
Nathan Le
Honey Patel
Jeremiah Penney
Emily Messenger

Revision History

Date	Version	Description	Author
03/04/26	1	Completed Logic view (5) and overview (5.1)	Honey Patel
3/07/26	1	Architecturally Significant Design Modules or Packages (5.2)	Honey Patel
3/09/26	1	Introduction (1), Purpose (1.1), Scope (1.2)	Jeremiah Penney
3/10/26	1	Interface Description (6) and Quality (8)	Emily Messenger
3/10/26	1	Overview (1.5), Architectural Representation (2), Architectural Goals and Constraints (3)	Nathan Le
3/12/26	1	Definitions, Acronyms, and Abbreviations (1.3) and References (1.4)	Honey Patel

©Smart Solve Calculations, 2026

Table of Contents

1. Introduction 4
   - 1.1 Purpose 4
   - 1.2 Scope 4
   - 1.3 Definitions, Acronyms, and Abbreviations 4
   - 1.4 References 5
   - 1.5 Overview 5
2. Architectural Representation 5
3. Architectural Goals and Constraints 6
4. Use-Case View 6
   - 4.1 Use-Case Realizations 6
5. Logical View 6
   - 5.1 Overview 6
   - 5.2 Architecturally Significant Design Packages 7
6. Interface Description 8
7. Size and Performance 8
8. Quality 8

Software Architecture Document

1. Introduction

This document is an overview of how the company (Smart Solve Calculations) uses software architecture for our Arithmetic Expression Evaluator system. The document explains how the system is structured and how many different components can work together to not only process but also evaluate any user's arithmetic expressions. Our document also talks about how the main design decisions that were used in our system and how each part contributes to making the system run as effectively as possible. This gives anyone a clear understanding of how systems are organized and how they will work.

1.1 Purpose

The overall purpose of this document is to describe the software architecture of our Arithmetic Expression Evaluator system. We explain how it is organized and show how the different components, like tokenizing, parsing, evaluating, and even error handling, can work with each other to process whatever the user inputs and will give them back a valid/correct result/output.

Our system is very diverse and can be helpful for different instructors and developers, or just anyone trying to solve an arithmetic expression. This document helps a variety of users understand how our team designed this system and how each part is working together to achieve one final output.

1.2 Scope

This document applies specifically to the Arithmetic Expression Evaluator system developed by the whole team. It focuses on how the system is shaped and how all the components are working in sync to achieve the goal of evaluating different arithmetic expressions entered by the user. The document covers the main parts of the system, including things like tokenizing, parsing, evaluating, and even error handling. It also explains how different parts will interact with each other. It also shows the design thought process and how are team built it to be as optimal and efficient as possible. This document only talks about the architecture and design of the system. This document does not include the full code or implementation details

1.3 Definitions, Acronyms, and Abbreviations

Arithmetic Expression – An expression that's a combination of operands and operators (+,-,/,*,%,**) that can be calculated using PEMDAS to obtain the result.

Expression Parsing– taking in a sequence of tokens and deciding whether that sequence is valid/runnable, while also focusing on precedence and parentheses.

Tokenizer – The first stage of the compiler in which the source code is translated into executable code.

Operator- symbol that represents the operation that will be performed. %,/,*,+,-, ** will be supported in this program.

Numeric constants- constants numbers in the program that are not changed

Operator precedence- The operators are evaluated using PEMDAS rules. (Parentheses, Exponentiation, Multiplication, Division, Addition, Subtraction)

Parentheses handling- The program can evaluate expressions within parentheses.

Unary operator- operator that works with just one value (+/-).

Command Line Interface- the software through which the user can input arithmetic expressions into the arithmetic expression evaluator.

1.4 References

1. Saiedian, Hossein. *EECS 348 Term Project: Arithmetic Expression Evaluator in C++*. Spring 2026. University of Kansas.

2. Course lecture materials and assignment specifications provided for EECS 348 – Spring 2026.

3. 02-Software-Requirements-Spec. March 2026. University of Kansas

1.5 Overview

This Software Architecture Document describes the high-level architecture of the Arithmetic Expression Evaluator in C++. The system is designed to accept arithmetic expressions from the user, parse them according to operator precedence and associativity rules, and evaluate them while handling invalid input and runtime errors such as division by zero. The evaluator supports the operators `+`, `-`, `*`, `/`, `%`, and `**`, along with unary `+` and `-`, numeric constants, and parentheses.

2. Architectural Representation

The architecture of the Arithmetic Expression Evaluator is represented as a pipe filter design implemented in C++. Its purpose is to separate user interaction, expression analysis, and computation into clear components, so the system is easier to understand, test, and maintain. This is a good fit for the project because the assignment explicitly emphasizes data transformation, testing, and alignment between design and implementation.

1. Lexer Package

This package is responsible for interacting with the user through a command-line interface. It accepts an arithmetic expression as input and decomposes it into separate tokens such as numbers, operators, and parentheses.

2. Parser Package

This package is responsible for analyzing the input expression and converting it into a structured internal form. It includes tokenization of numbers, operators, and parentheses, followed by syntactic analysis based on precedence, associativity, and grouping rules. This layer ensures that expressions such as nested parentheses and unary operators are interpreted correctly before evaluation begins.

3. Evaluator Package

This package computes the result from the parsed expression. It applies arithmetic operations in the proper order, including left-to-right evaluation for `+`, `-`, `*`, `/`, and `%`, and right-to-left evaluation for `**`. It also handles unary

operators and validates runtime conditions such as division or modulo by zero.

4. Error Handling Package

This package works across the system to detect and report invalid expressions, mismatched parentheses, illegal characters, missing operands, and mathematically invalid operations. Error handling is architecturally significant because robustness is a major grading criterion in the project.

User Input → Tokenizer → Parser → Evaluator → Error Handler → Output

3. Architectural Goals and Constraints

The main goal of this system is to correctly parse and evaluate arithmetic expressions using proper operator precedence, associativity, and parentheses. The program must support operators such as +, -, *, /, %, and **, as well as unary operators, and produce accurate results for valid inputs.

Another goal is to handle errors effectively, including invalid expressions, unmatched parentheses, and division by zero, while providing clear error messages to the user. The system should also be easy to maintain and test, so a modular and object-oriented design in C++ is used.

There are several constraints that affect the architecture. The program must be implemented in C++ and use a command-line interface for user input and output. It is assumed that inputs are integers, and the system must follow the required operator's precedence and associativity rules.

4. Use-Case View

4.1 Use-Case Realizations

5. Logical View

The architecture of the design model is outlined below. It includes the high-level structure needed to eventually code the system. Components, connections, and the constraints are included. The components highlight the major pieces of the system, connections outline how the components interact with each other, and the constraints include the rules that guide the design. This modular design ensures that the system is maintainable, testable, and easy to grow.

5.1 Overview

The system will be organized using the pipe-filter architectural style. The input data will flow through sequences and go through data transformation such as tokenizing, parsing, evaluation, and error handling. This modular design will reduce complexity by breaking each step into smaller steps.

5.2 Architecturally Significant Design Modules or Packages

1. **Lexer Package**: This package will decompose input expressions into separate tokens such as numbers, operators, and parentheses.

Class 1: Tokenizer: Input expression is broken down into tokens

Responsibilities: Identifies numbers/operators

2. **Parser Package**: This package will take the tokens and ensure the expression was valid. The tokens will be organized based on operator precedence.

Class 1: Parser: Parses through the tokens and organizes them into an expression through operator precedence.

Responsibilities: Handles operator precedence, ensures valid syntax

3. **Evaluator Package**: This package will take the expression and calculate the result. The operations will be applied using PEMDAS.

Class 1: Evaluator: Expression is computed by applying arithmetic operations with operator precedence.

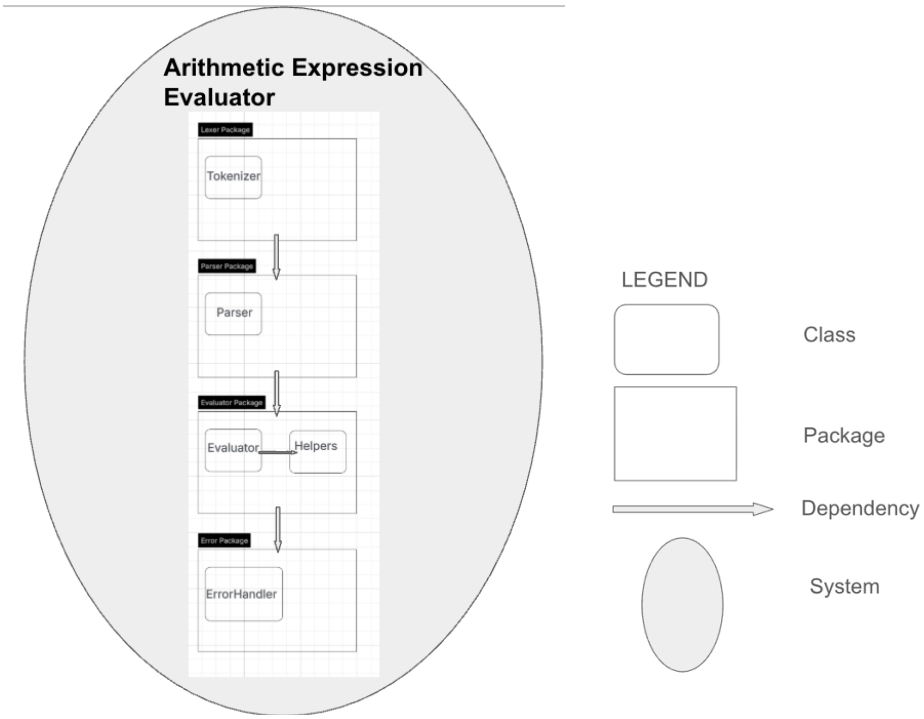
Responsibilities: Handles operator precedence, perform arithmetic operations

Class 2: Helpers: Holds helper functions for performing arithmetic operations and checking operator precedence

4. **Error Handler Package**: This package will detect errors in the input and errors that arise during the computation. Error messages will be returned to the user.

Class 1: Error handler: Handles errors during processing and computation and returns error message.

Responsibilities: Detect invalid input, handle errors, return error message to users



6. Interface Description

We will use a Command Line Interface (CLI) that allows users to perform basic arithmetic operations through text-based input.

Interface Format:

- The program runs in a terminal window.
- The system prompts the user to enter:
    1. The first number
    2. An operator
    3. The second number
- The user presses Enter to submit input.

Valid Inputs:

- Integers or decimal numbers
- Operators: +, -, /, %, or **
- One operator per expression
- No division by zero

Resulting Outputs:

- If the input is valid, the program will display the computed result.
- Otherwise, the system will display an error message (e.g., “Error: Invalid input”) and prompt the user to try again.

7. Size and Performance

8. Quality

Reliability: The system will include input validation and error handling to prevent invalid operations (e.g., division by zero, malformed expressions). Clear error messages will ensure predictable and stable behavior under incorrect input conditions.

Usability: The command-line interface allows for a simple usage process for the users in which they can input a arithmetic expression and receive a result. Error messages will be displayed and allow for input re-entering.

Extensibility: The architecture will separate input handling, computation logic, and output display. This modular design will allow additional features (ex. exponentiation, logarithmic functions, or calculation history) to be added with minimal modification to existing components.

Portability: As a command-line application written in a standard programming language (e.g., Python, Java, or C++), the calculator will run on multiple operating systems with minimal or no modification.